

This is the definitive Security & Authorization Architecture for V1 of the DTE Platform. As a self-hosted platform where the customer retains absolute sovereignty over their infrastructure, data, and cryptographic material, the security architecture is predicated on strict separation of duties, cryptographic non-repudiation, and tamper-evident auditability. This document defines the boundaries, constraints, and operational security models required to ensure full regulatory compliance and legal defensibility.

SECTION 1 — Security Philosophy

To satisfy enterprise security standards and Ministry of Finance (MH) regulatory audits, the architecture enforces the following immutable principles:

- **Customer Sovereignty & Zero-Knowledge Vendor:** The customer owns the legal identity, the data, and the liability. The software vendor distributes the software and regulatory rules but has zero access to customer data, Private Keys, or MH credentials.
- **Cryptographic Non-Repudiation:** The platform mathematically proves what was issued, when, and by whom. The customer's exclusive custody of the signing key ensures that all generated evidence is legally binding and incontrovertible.
- **Defensible Auditability (Tamper-Evidence):** The platform operates under the assumption that internal actors or compromised system processes may attempt to alter history. The hash-chained AuditTrail and cryptographically hashed EvidenceBundle render tampering mathematically detectable.
- **Immutability over Deletion:** Security-relevant data is never destroyed. Deprecation and soft-deletes are utilized for operational data; evidence and audit data are physically prevented from deletion at the database boundary.
- **Least Privilege Execution:** The application assumes minimal necessary rights. Database connections used for operational issuance lack DELETE permissions on evidence and audit tables.

SECTION 2 — Identity Model

The Identity Model in V1 is strictly scoped to human actors operating within the self-hosted deployment.

- **Human Actors:**
 - **Owner:** The administrative and governance identity.
 - **Biller:** The operational business identity.
- **Future Service Actors:** Programmatic identities (Service Accounts) are deferred to V1.5+ but anticipated by ensuring every identity resolves to a globally unique UUID in the `app_user` table.
- **Authentication Principles:** Identity must be strongly bound to a cryptographic session token upon successful credential verification. The application relies on the system's internal identity store for V1.
- **Identity Lifecycle:** * *Creation:* Provisioned by an existing Owner (or via secure bootstrap for the first Owner).
 - *Active:* Allowed to establish sessions.
 - *Deactivated:* Access revoked immediately. Identities are **never** hard-deleted to ensure

the historical AuditTrail maintains referential integrity to the actor.

- **Account Lifecycle:** Sessions are logically separated from the identity. A deactivated identity automatically invalidates all active sessions.

SECTION 3 — Authorization Model

V1 utilizes a strict Role-Based Access Control (RBAC) model overlaying all API commands.

- **Role Model:**
 - **Owner:** Possesses global permissions to configure the issuer_configuration, manage app_user assignments, update master catalogs, declare ContingencyEpisodes, and perform all operational billing tasks.
 - **Biller:** Restricted to operational tasks: drafting, validating, signing, transmitting, and invalidating FiscalDocuments.
- **Permission Model:** Permissions are implicitly mapped to roles in V1. There is no custom permission matrix.
- **Policy Model:** Because V1 is a single-issuer deployment, policies are globally scoped. (In future multi-tenant scenarios, policies will scope roles to specific Establishments).
- **Operational Restrictions & Rationale:**
 - *Why Billers cannot manage Issuer Configuration:* The issuer_configuration contains pointers to MH credentials and signing certificates. Compromise of this configuration compromises the legal identity of the corporation.
 - *Why Billers cannot declare Contingency:* A ContingencyEpisode is a legal exception regime with strict MH compliance windows. Entering this regime requires governance oversight (Owner) to accept the compliance risk of offline issuance.

SECTION 4 — Trust Boundaries

Explicit boundaries isolate risk and define security responsibilities.

- **User Trust Boundary (Browser $\rightarrow$ Application):**
 - *Responsibility:* Protects session integrity and data-in-transit.
 - *Mechanism:* TLS 1.2+ mandatory. Authentication via secure, HTTPOnly, SameSite cookies or short-lived JWTs.
- **Application Trust Boundary (Application $\rightarrow$ Database):**
 - *Responsibility:* Protects data-at-rest and enforces immutability.
 - *Mechanism:* The application uses scoped database roles. The role used by the web application has INSERT and SELECT rights to the audit_entry and evidence_artifact tables, but UPDATE and DELETE privileges are physically revoked at the PostgreSQL level.
- **Signing Trust Boundary (Application $\rightarrow$ Key Custodian/Vault):**
 - *Responsibility:* Protects the Private Key.
 - *Mechanism:* The application NEVER loads the .pfx or private key directly into application memory. It uses the certificate_reference to request a cryptographic signature from the customer's secure local vault, HSM, or OS-level keystore.
- **MH Trust Boundary (Application $\rightarrow$ Ministry of Finance API):**
 - *Responsibility:* Authenticated, encrypted communication with the regulator.
 - *Mechanism:* Mutual TLS (if required by MH) and bearer-token authentication using the

dynamically requested MH API token.

SECTION 5 — Certificate Ownership Model

The Digital Certificate represents the legal entity. The software is merely an agent acting on its behalf.

- **What the Platform Knows:** The Public Key, Subject Name, Valid From/To dates, Thumbprint, and the certificate_reference (URI/Pointer to the vault).
- **What the Platform MUST NEVER Know:** The Private Key material.
- **Certificate Registration:** An Owner provides the certificate_reference and the public certificate chain into the issuer_configuration.
- **Certificate Expiration:** The platform actively monitors the expiration date of the public certificate and triggers UI alerts to Owners starting 30 days prior to expiration.
- **Certificate Replacement:** An Owner updates the certificate_reference. The platform logs a critical CONFIG_CHANGED audit event.
- **Certificate Revocation:** If the certificate is revoked by the CA or MH, the MH API will return terminal rejections (RECHAZADO). The platform records these in the TransmissionRecord.

SECTION 6 — Secret Management Model

All operational secrets are owned by the customer and governed by their internal IT policies.

- **MH Credentials (API Passwords/Tokens):**
 - *Ownership:* Customer Owner.
 - *Storage:* Kept in a secure local vault/environmental variables; the database holds a credentials_reference.
 - *Rotation:* Customer rotates via MH portal, then updates the vault.
- **Signing Credentials (Key PINs/Passwords):**
 - *Ownership:* Customer Owner.
 - *Storage:* Managed exclusively by the Customer's HSM or Secure Vault. The application submits payloads to the vault API for signing.
- **Encryption Keys (Database/Disk Encryption):**
 - *Ownership:* Customer Infrastructure Team.
 - *Storage:* Managed by the underlying cloud provider or hypervisor (Transparent Data Encryption).
- **Operational Secrets (JWT Secrets, DB Passwords):**
 - *Ownership:* Customer Infrastructure Team.
 - *Storage:* Injected into the application environment at runtime. Rotated via standard DevSecOps pipelines.

SECTION 7 — Authentication Architecture

- **Login Lifecycle:** User submits credentials → Application validates hash → Application issues a cryptographically signed session token (JWT) containing the actor_identity and actor_role.

- **Session Lifecycle:** Sessions have a hard expiration (absolute timeout) and an idle timeout. If an Owner changes a user's role or deactivates them, a token-revocation list (or short JWT expiry) ensures the session is terminated within minutes.
- **Password Lifecycle:** Passwords are never stored in plaintext. They are hashed using modern, memory-hard algorithms (e.g., Argon2id or Bcrypt). Minimum complexity requirements are enforced by the platform.
- **Recovery Lifecycle:** In V1 (no email delivery integration required), password recovery is administrative. An Owner can reset a Biller's password. If the sole Owner loses access, a secure CLI/infrastructure-level script is required to bootstrap access, demanding server-level physical/SSH security.

SECTION 8 — Authorization Architecture (Command Matrix)

Every command validates the JWT claims before execution.

Command	Required Role	Authorization Check / Restrictions	Audit Trigger
CreateDraftDocument	Owner, Biller	Validates actor is Active.	DOCUMENT_DRAFTED
ValidateDocument	Owner, Biller	Validates Document is Draft.	DOCUMENT_VALIDATED
SignDocument	Owner, Biller	Validates Document is Validated.	DOCUMENT_SIGNED
TransmitDocument	Owner, Biller	Validates no active Contingency exists.	TRANSMISSION_ATTEMPTED
RequestInvalidation	Owner, Biller	Validates target is Accepted.	INVALIDATION_REQUESTED
OpenEpisode (Contingency)	Owner Only	Rejects Biller. Validates no open episode.	CONTINGENCY_STARTED
TransmitNotification	Owner Only	Rejects Biller. Validates episode is Open.	CONTINGENCY_NOTIFIED

CreateCounterpart y	Owner Only	Rejects Biller.	CATALOG_UPDATE D
ConfigureIssuer	Owner Only	Rejects Biller.	CONFIG_CHANGE D

SECTION 9 — Audit Attribution Model

Defensibility requires that every business action is non-repudiable.

- **Actor Attribution:** The actor_identity (UUID) and actor_role are extracted directly from the trusted JWT session, never from client-provided API payloads.
- **Command Attribution:** The precise action type (e.g., DOCUMENT_SIGNED) is hardcoded into the API command handler.
- **Evidence Linkage:** The AuditEntry records the TargetIdentity (e.g., the Document_ID). Auditors can traverse from the AuditEntry $\rightarrow$ FiscalDocument $\rightarrow$ EvidenceBundle to retrieve the exact cryptographic payload that was generated as a result of the action.
- **Hash Chaining:** The CurrentHash of the audit entry embeds the actor_identity, ensuring an attacker cannot swap attribution without breaking the cryptographic chain of the entire ledger.

SECTION 10 — Operational Security

- **Failed Login Handling:** The platform tracks failed authentication attempts.
- **Account Lockout:** After a configurable number of consecutive failed attempts (e.g., 5), the account is temporarily locked to prevent brute-force attacks.
- **Suspicious Activity:** Invalid JWT signatures, expired tokens, or unauthorized role escalations are denied and logged as SECURITY_EVENT in standard application logs (distinct from the business Audit Trail).
- **Privileged Operations:** Operations like ConfigureIssuer or uploading a RegulatoryBundle demand re-authentication (or validation of a highly fresh token) to prevent session hijacking from compromising the platform.

SECTION 11 — Regulatory Security Considerations

- **Issuer Accountability:** The IssuerSnapshot physically copies the legally binding NIT/NRC into the document at the exact moment of validation, proving the identity under which the transaction was legally executed.
- **Signature Accountability:** Because the customer holds the private key in their vault, any document bearing a valid signature mathematically proves the customer's intent to issue. The vendor has zero deniability or capability to forge this.
- **Transmission Accountability:** The TransmissionRecord acts as a black-box flight recorder. It captures the exact byte payload sent and the exact byte payload received

from the MH, preventing disputes over "what the API returned" during an audit.

- **Invalidation Accountability:** The MH schema requires a human "Responsible" to authorize invalidations and contingencies. The platform maps the authenticated app_user's metadata to these mandated fields, legally binding the human operator to the regulatory exception.

SECTION 12 — Security Event Model

Logs are segmented by purpose and retention requirements.

- **Security Events (App Logs):** Login successes/failures, lockouts, blocked access attempts, TLS errors. *Retention:* Managed by customer IT (e.g., forwarded to a SIEM), typically 1 year.
- **Operational Events (App Logs):** Service startup/shutdown, database connection drops, RegulatoryBundle ingestion events. *Retention:* Customer IT, typically 30-90 days.
- **Audit Events (Database audit_trail):** All commands listed in Section 8. *Retention:* Indefinite (Minimum 10+ years per tax retention laws). Backed up transactionally with the core database.

SECTION 13 — Future Security Expansion

The V1 architecture provides explicit "seams" to support V1.5/V2 enterprise features without redesigning the security core:

- **Service Accounts:** The app_user table structure accepts programmatic identities. JWTs can be issued as long-lived API keys, restricted by a new Policy engine layer overlaid on the API endpoints.
- **External Identity Providers (IdP):** The Authentication Architecture can bypass local password verification and accept SAML/OIDC assertions from Azure AD/Okta, subsequently mapping the IdP groups to the internal Owner/Biller roles and issuing the internal JWT.
- **Approval Workflows:** The Draft $\rightarrow$ Validated transition can be intercepted. A Biller's request to validate high-value documents can route to an ApprovalRequest entity, requiring an Owner's cryptographic assertion before the state machine proceeds.